Technical Report — Sokoto Rainfall Cover

Capstone Project: Technical Implementation & Development Progress

Chinemerem Judith Ugbo · Supervisor: Dr Aaron Izang

Technical Report: Sokoto Rainfall Cover

Repository: github.com/Phinatech/Capstone-project · *Live:* sokoto-cover.pages.dev · *Chain:* Ethereum Sepolia

1. Scope Alignment & Feature Validation

The approved scope proposed a blockchain-based weather-index insurance system in which (1) rainfall is measured by multiple independent sources, (2) policies are sold and settled by smart contract without a claims process, and (3) farmers, field agents and administrators manage cover through a web application. The delivered system matches that scope; one planned element (SMS delivery) is implemented but optional in configuration, and fiat settlement is deliberately deferred (it requires a licensed payment provider).

Planned Feature	Intended Purpose	Current Status	Evidence
Smart contracts: policy sale, premium, payout reserve	Sell cover; guarantee payout trustlessly	**Completed** · M1 · Contracts	WeatherIndexInsurance.sol; 56 tests; Policy #0 bought & paid on Sepolia
Multi-source oracle with reputation-weighted aggregation	One source cannot decide a claim	**Completed** · M2 · Oracle	OracleAggregator.sol; 3 live sources; sim. Fig. 07
Automated oracle relayer + settlement keeper	Report rainfall and pay farmers unattended	**Completed** · M2 · Oracle	relayer.ts, settlement.ts; Policy #0's live payout ran

Planned Feature	Intended Purpose	Current Status	Evidence
			autonomously
Farmer portal (buy cover, monitor, payouts)	Self-service cover with a wallet	**Completed** · M3 · Portals	/farmer/*; screenshots 02, 03, 04; route tests
Field-agent portal (sell cover, naira receipts)	Reach farmers without wallets	**Completed** · M3 · Portals	/agent/*; receipts verified server-side against the chain
Admin console (policies, farmers & agents, analytics)	Operate and audit the scheme	**Completed** · M3 · Portals	/admin/*; screenshots 05, 06
Oracle research tools (consensus simulator, backtest)	Study/ demonstrate consensus design	**Completed** · M4 · Research	/admin/backtest; 121 historical dekads (backtest-results.md)
Notifications (in-app, email codes, SMS/Telegram)	Tell farmers about purchases/payouts	**Completed (SMS optional)** · M4 · Research	real event notifications; email codes; SMS via SMS_PROVIDER
Testing & security audit	Validate and harden the system	**Completed** · M5 · Validation	292 tests; security-audit.md, fixes pinned by regressions
Deployment (public site + testnet)	Working product, end to end	**Completed** · M5 · Validation	sokoto-cover.pages.dev + Render + Sepolia
Fiat settlement (naira off-ramp)	Pay farmers in naira	**Not implemented — justified** · M6 · Future	Needs a licensed provider; receipts record the naira leg; future work

Scope changes: none dropped for convenience. The naira leg was designed as receipted off-chain settlement (a licensed off-ramp is beyond a student project), and deployment widened from a local chain to live Sepolia — adding real wallet integration, RPC resilience and network-safety work.

2. Technical Product Demonstration

The walkthrough video (linked in the README) demonstrates, in order:

A. Core functionalities — policy purchase as agent and as farmer, the farmer dashboard and payouts ledger, the admin's *oracle evaluation* view, the consensus simulator corrupting one source, and the backtest. **B. Technical requirements** — every screen wired end-to-end: React/TypeScript → Express/TypeScript API → Firebase → ethers.js → Solidity on Sepolia, fed by three real rainfall APIs. **C. Testing with inputs** — wrong premium, closed window, insufficient pool capacity, wrong wallet network, invalid codes, role escalation: each shown refusing safely with a plain-language error. **D. Testing strategies** — unit/functional and integration testing (§3.3). **E. Performance & compatibility** — measured latency and bundle sizes (§3.4), desktop and mobile layouts, Chromium browsers.

3. Code Quality & Technical Implementation

3.1 OOP principles and architecture

The codebase is organised by layer — `blockchain/` (contracts), `backend/` (API, relayer, stores), `frontend/` (portals, providers) — with clear separation of concerns and no monolithic scripts.

- **Abstraction & encapsulation.** Each store (`createUserStore`, `createSessionStore`, `createAgentStore`, `createAuthCodeStore`) hides its persistence behind one `StorePersistence` interface; the application does not know or care whether records live in Firestore, a JSON file or an in-memory map. The same is true of the policy layer (`usePolicies()` hides chain vs simulated providers).
- **Polymorphism & strategy.** Persistence backends are interchangeable strategies — production (Firebase), development (files), tests (fakes) — chosen once at start-up. The consensus simulator runs three aggregation strategies (single-source, equal, reputation-weighted) behind one interface.
- **Dependency injection.** `createAuthRouter({ users, sessions, authCodes })` receives its collaborators, so end-to-end tests exercise the real router over HTTP against isolated stores.
- **Inheritance & reusable components.** `ErrorBoundary` extends `React.Component` once and protects every route; typed `interface` contracts (`User`, `Policy`, `StorePersistence`) are implemented by interchangeable parts. Shared reusable components (`PolicyBrowser`, `PageHeader`, `Avatar`) and pure

utilities (utils/oracle.ts, utils/currency.ts) compose the frontend; the Context tree (theme → i18n → preferences → auth → policy) layers cross-cutting behaviour without duplication.

3.2 Algorithms and problem-solving

- **Reputation-weighted consensus.** Dekad rainfall is a weighted average over reporting sources; weights come from peer-agreement scores, so a corrupt source loses weight immediately and historically. The simulator shows a corrupted feed moving the aggregate by **+9%** under reputation weights versus **+59%** under equal weights.
- **Out-of-sample backtest.** Weights are learned from inverse-MSE on all years *except* the scored year, with 2,000-resample bootstrap confidence intervals — avoiding a fit to the test data (docs/backtest-results.md).
- **Fixed-point time-series arithmetic.** Rainfall is integer hundredths of millimetres; dekads are period ids (YYYYMMD); windows sum exactly their period set — settlement is exact and auditable.
- **Concurrency-safe persistence.** Stores serialise writes through a promise chain and temp-file-and-rename; the relayer fetches sources with bounded concurrency (mapLimit) to respect free tiers.
- **Security algorithms.** scrypt password hashing, constant-time API-key comparison, HMAC-hardened session ids, single-use attempt-capped verification codes, and per-hour rate limits — each with regression tests.

3.3 Testing and validation (two strategies, with results)

Strategy 1 — Unit and functional testing (chosen because the contract invariants *are* the product's fairness guarantees). 56 Solidity tests (Hardhat) including a fuzz test over pool reserves, plus 203 backend unit tests (node:test) and 20 frontend tests (Vitest). Coverage includes: premium = 10% of payout rounded up, sales the pool cannot cover revert, payouts happen exactly once and only when rainfall < threshold, and every audit finding pinned by a named regression test. *Result:* 292 green; the suite caught real bugs (a GET-vs-POST resend route, a seed-time password leak, tests leaking into production data — all fixed and pinned).

Strategy 2 — Integration and end-to-end testing (chosen because the guarantees span HTTP, storage, wallets and chain). Mocha drives the contracts through ethers.js exactly as the app does (13 scenarios); auth routes run against a real Express server with real cookie jars; the deployed system was exercised live (the first policy bought and paid out autonomously). *Result:* verified in

production; failures traced to wallet infrastructure (stale RPC state, wrong network) and hardened — gas pre-estimated on the app's own RPC, account switches followed, wrong-chain signing refused.

Validation: invalid codes, wrong premiums, expired windows, role-escalation payloads and unreachable RPCs each produce a plain-language, actionable error.

3.4 Performance and compatibility

Measurement	Result	Conditions
REST API latency	0.44–0.82 s per read	Render free tier, warm
Frontend build	8.3 s; 1.4 MB JS across lazy chunks	Vite, code-split
Backend test suite	~6.5 s, 203 tests	Node 22
Layout	Responsive desktop → 390 px mobile	captures 02, 09
Browsers	Chrome/Edge, macOS & Linux	no browser-specific APIs beyond EIP-1193 wallet injection
Chain	Ethereum Sepolia (11155111), MetaMask	hardened vs stale RPCs/wrong network

The farmer side targets low-power hardware: the mobile layout renders at 390 px, naira figures degrade gracefully when FX providers are unavailable, and chain reads tolerate RPC hiccups (retry wrappers, cached reads).

3.5 Engineering quality

Focused imperative commit history, consistent naming, comments at decision points (why, not what), strict TypeScript in both workspaces, and a documentation set: README, deployment guide, security audit, debugging journal, licences. The five most instructive production bugs are preserved in docs/debugging-journal.md as an engineering record.

Supporting evidence: docs/screenshots/ (10 captioned figures), docs/backtest-results.md, docs/security-audit.md, docs/debugging-journal.md, and the live deployment.